

Lab7 Report: Containerization with Docker (English Version)

1. Experiment Purpose

1. Build a customized Nginx Docker image with a static HTML webpage via writing Dockerfile.
2. Master container port mapping between host and container, verify immutable infrastructure feature of Docker images.
3. Learn container resource monitoring, multi-instance deployment and port conflict rules.
4. Archive all experiment source files into GitHub repository by Git commands.

2. Experimental Environment

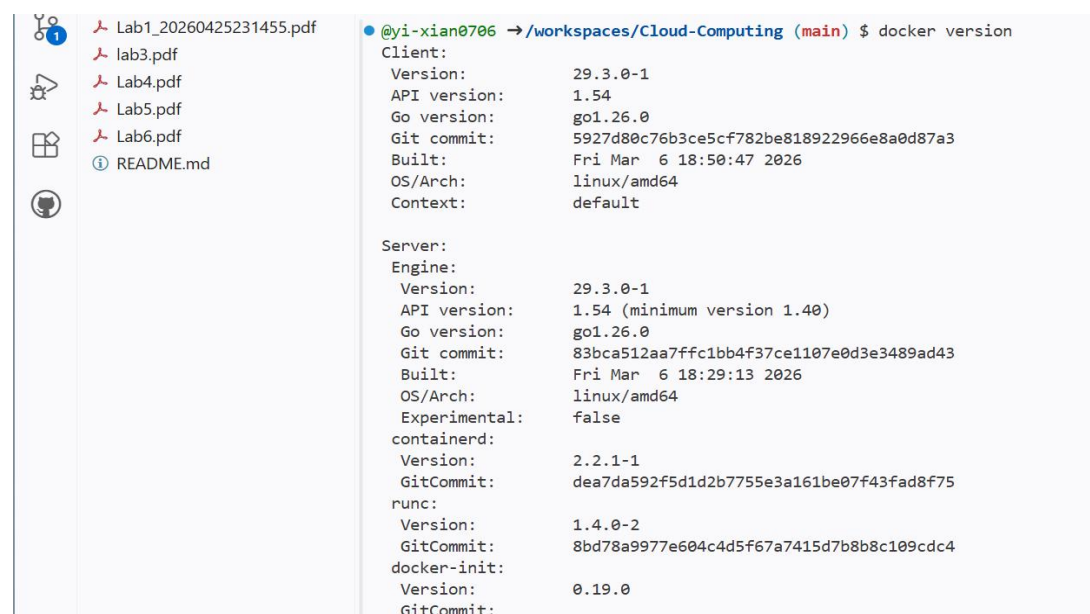
Cloud environment: GitHub Codespaces (built-in Docker & Git service)

Source files: `index.html` (static web page), self-defined `Dockerfile`

3. Detailed Experimental Procedures & Results

Step 1: Pre-check Docker environment

Run command `docker version` in terminal to verify Docker client and daemon work normally.



```
@yi-xian0706 →/workspaces/Cloud-Computing (main) $ docker version
Client:
Version:      29.3.0-1
API version:  1.54
Go version:   go1.26.0
Git commit:   5927d80c76b3ce5cf782be818922966e8a0d87a3
Built:        Fri Mar  6 18:50:47 2026
OS/Arch:      linux/amd64
Context:      default

Server:
Engine:
Version:      29.3.0-1
API version:  1.54 (minimum version 1.40)
Go version:   go1.26.0
Git commit:   83bca512aa7ffc1bb4f37ce1107e0d3e3489ad43
Built:        Fri Mar  6 18:29:13 2026
OS/Arch:      linux/amd64
Experimental: false
containerd:
Version:      2.2.1-1
GitCommit:    dea7da592f5d1d2b7755e3a161be07f43fad8f75
runc:
Version:      1.4.0-2
GitCommit:    8bd78a9977e604c4d5f67a7415d7b8b8c109cdc4
docker-init:
Version:      0.19.0
GitCommit:
```

Step 2: Create Dockerfile and build custom image

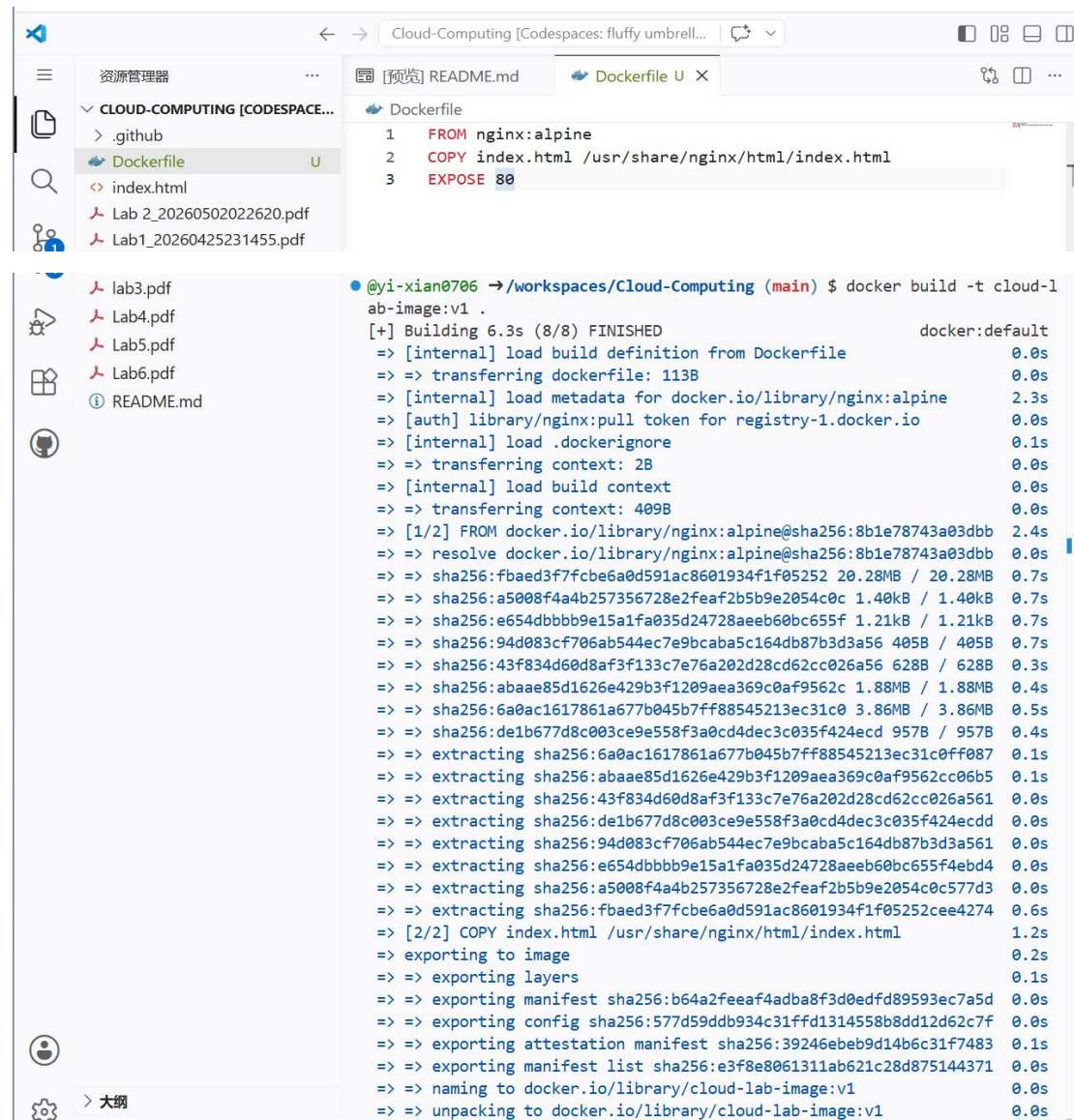
1. Create Dockerfile in project root folder with content below:

```
dockerfile
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

2. Build image with command:

```
bash
docker build -t cloud-lab-image:v1 .
```

The terminal outputs final success prompt indicating image `cloud-lab-image:v1` is generated successfully.



The screenshot shows a VS Code editor with a workspace named 'Cloud-Computing [Codespaces: fluffy umbrella...]'. The left sidebar shows a file explorer with a folder named 'CLOUD-COMPUTING [CODESPACE...]' containing files like '.github', 'Dockerfile', 'index.html', and two PDF files. The main editor area shows the 'Dockerfile' with the following content:

```
1 FROM nginx:alpine
2 COPY index.html /usr/share/nginx/html/index.html
3 EXPOSE 80
```

Below the editor, a terminal window shows the output of the command `docker build -t cloud-lab-image:v1 .`. The output indicates a successful build of the image `cloud-lab-image:v1`.

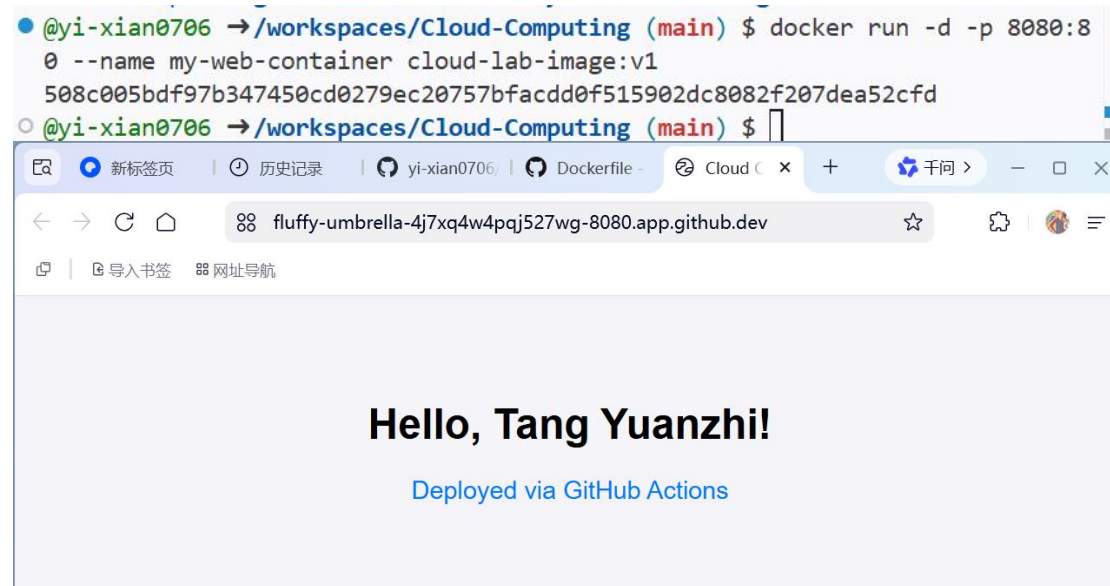
```
@yi-xian0706 → /workspaces/Cloud-Computing (main) $ docker build -t cloud-lab-image:v1 .
[+] Building 6.3s (8/8) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 113B                             0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine  2.3s
=> [auth] library/nginx:pull token for registry-1.docker.io    0.0s
=> [internal] load .dockerignore                                0.1s
=> => transferring context: 2B                                    0.0s
=> [internal] load build context                                0.0s
=> => transferring context: 409B                                  0.0s
=> [1/2] FROM docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb 2.4s
=> => resolve docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb 0.0s
=> => sha256:fbaed3f7fcbe6a0d591ac8601934f1f05252 20.28MB / 20.28MB 0.7s
=> => sha256:a5008f4a4b257356728e2feaf2b5b9e2054c0c 1.40kB / 1.40kB 0.7s
=> => sha256:e654dbbbb9e15a1fa035d24728aeeb60bc655f 1.21kB / 1.21kB 0.7s
=> => sha256:94d083cf706ab544ec7e9bcaba5c164db87b3d3a56 405B / 405B 0.7s
=> => sha256:43f834d60d8af3f133c7e76a202d28cd62cc026a56 628B / 628B 0.3s
=> => sha256:abaae85d1626e429b3f1209aea369c0af9562c 1.88MB / 1.88MB 0.4s
=> => sha256:6a0ac1617861a677b045b7ff88545213ec31c0 3.86MB / 3.86MB 0.5s
=> => sha256:de1b677d8c003ce9e558f3a0cd4dec3c035f424ecd 957B / 957B 0.4s
=> => extracting sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff087 0.1s
=> => extracting sha256:abaae85d1626e429b3f1209aea369c0af9562c06b5 0.1s
=> => extracting sha256:43f834d60d8af3f133c7e76a202d28cd62cc026a561 0.0s
=> => extracting sha256:de1b677d8c003ce9e558f3a0cd4dec3c035f424ecd 0.0s
=> => extracting sha256:94d083cf706ab544ec7e9bcaba5c164db87b3d3a561 0.0s
=> => extracting sha256:e654dbbbb9e15a1fa035d24728aeeb60bc655f4ebd4 0.0s
=> => extracting sha256:a5008f4a4b257356728e2feaf2b5b9e2054c0c577d3 0.0s
=> => extracting sha256:fbaed3f7fcbe6a0d591ac8601934f1f05252cee4274 0.6s
=> [2/2] COPY index.html /usr/share/nginx/html/index.html      1.2s
=> exporting to image                                           0.2s
=> => exporting layers                                           0.1s
=> => exporting manifest sha256:b64a2feef4adba8f3d0edfd89593ec7a5d 0.0s
=> => exporting config sha256:577d59ddb934c31ffd1314558b8dd12d62c7f 0.0s
=> => exporting attestation manifest sha256:39246eb9d14b6c31f7483 0.1s
=> => exporting manifest list sha256:e3f8e8061311ab621c28d875144371 0.0s
=> => naming to docker.io/library/cloud-lab-image:v1            0.0s
=> => unpacking to docker.io/library/cloud-lab-image:v1          0.0s
```

Step3: Launch container and access webpage

Execute running command for container deployment:

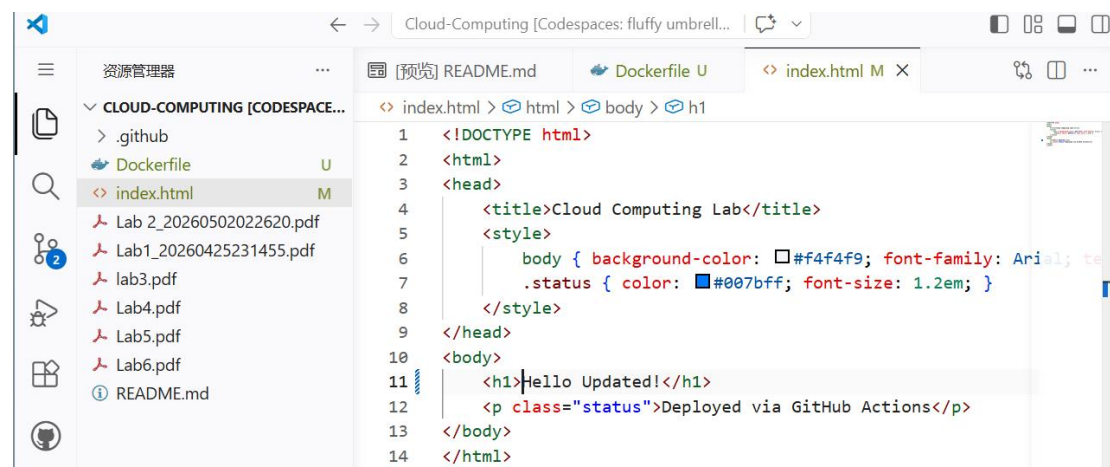
```
bash
docker run -d -p 8080:80 --name my-web-container cloud-lab-
image:v1
```

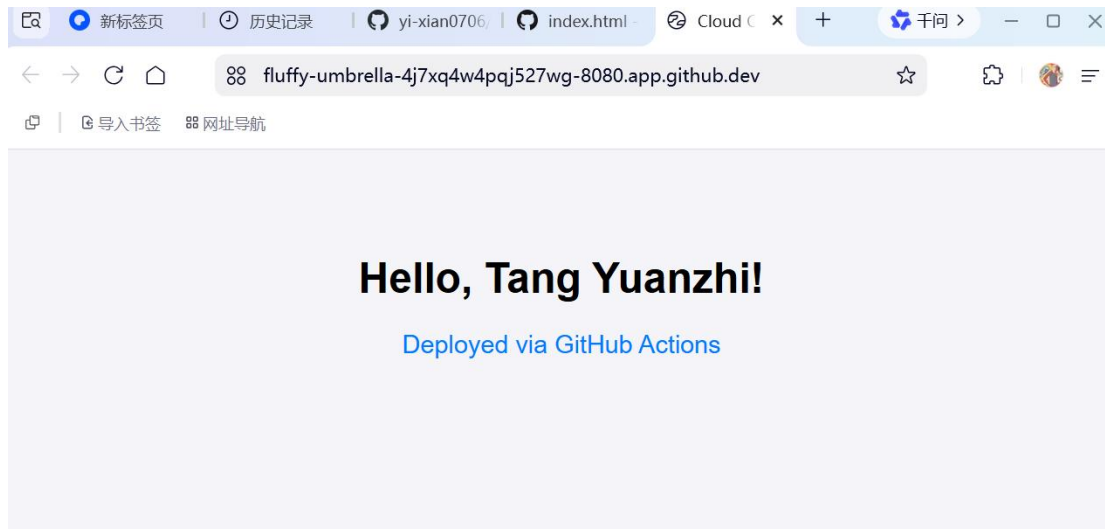
Open port 8080 from Codespaces port panel, browse the static HTML page successfully on a new browser tab.



Step4: Verify immutability of Docker image

1. Edit local `index.html`, change page content from Hello, Tang Yuanzhi! to Hello Updated! and save modification.
2. Refresh browser page on port 8080: webpage content keeps original text without any change.





Step5: Inspect internal files inside running container

Log into container's shell environment with the command:

```
bash
docker exec -it my-web-container sh
```

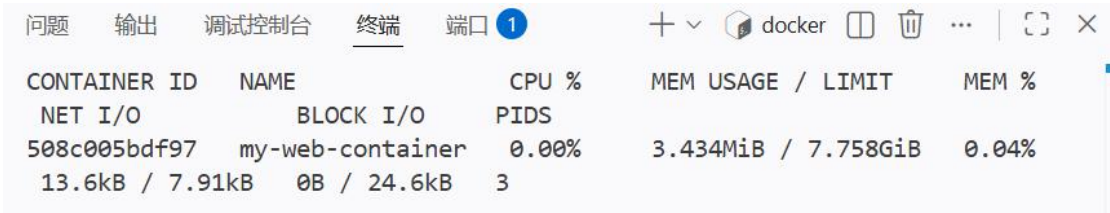
Run `ls /usr/share/nginx/html` and `cat /usr/share/nginx/html/index.html` sequentially. The printed HTML content inside container is still the original unmodified version, proving file system isolation between container and host. Type `exit` to quit container shell.

```
● @yi-xian0706 → /workspaces/Cloud-Computing (main) $ docker exec -it my-web-
  container sh
  / # ls /usr/share/nginx/html
  50x.html    index.html
  / # cat /usr/share/nginx/html/index.html
  <!DOCTYPE html>
  <html>
  <head>
    <title>Cloud Computing Lab</title>
    <style>
      body { background-color: #f4f4f9; font-family: Arial; text-align: center; padding-top: 50px; }
      .status { color: #007bff; font-size: 1.2em; }
    </style>
  </head>
  <body>
    <h1>Hello, Tang Yuanzhi!</h1>
    <p class="status">Deployed via GitHub Actions</p>
  </body>
  </html>
  / # exit
○ @yi-xian0706 → /workspaces/Cloud-Computing (main) $
```

Step6: Monitor container runtime resource

Run `docker stats` to check real-time CPU and memory usage of the Nginx container.

Experimental data: Container only occupies about 3.434MiB memory with nearly 0% CPU usage, which reflects Docker's lightweight advantage compared with traditional virtual machines. Press `Ctrl+C` to exit monitoring panel.



CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %
508c005bdf97	my-web-container	0.00%	3.434MiB / 7.758GiB	0.04%
NET I/O	BLOCK I/O	PIDS		
13.6kB / 7.91kB	0B / 24.6kB	3		

Step7: Multi-container deployment and port conflict test

1. Start second container with unused port 8081:

```
bash
docker run -d -p 8081:80 --name second-container cloud-lab-image:v1
```

Two available ports (8080 & 8081) appear on port panel, both links open identical webpage content.

2. Try starting third container binding occupied port 8080:

```
bash
docker run -d -p 8080:80 --name third-container cloud-lab-image:v1
```

System throws error `port is already allocated`, verifying one host port can only bind one single container process.



端口	转发地址	正在运行的进程	可见性	源
8080	https://fluffy-um...	/usr/libexec/dock...	Private	自动转发
8081	https://fluffy-um...	/usr/libexec/dock...	Private	自动转发

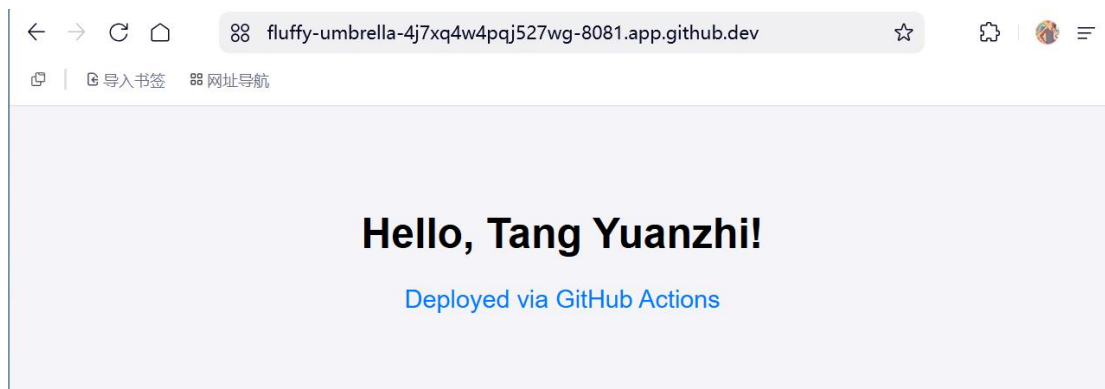
添加端口


```

● @yi-xian0706 → /workspaces/Cloud-Computing (main) $ docker run -d -p 8081:8081 --name second-container cloud-lab-image:v1
a84d8957bf2592614e07e2892c530e954569ec74fafd5e8a3f7d995792a8c48a
○ @yi-xian0706 → /workspaces/Cloud-Computing (main) $ docker run -d -p 8080:8080 --name third-container cloud-lab-image:v1
1ab380cfc3e101c3189660e2e1f9da797c172a3d56de64c4421537f5ee156113
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint third-container (2c31223a732bdccbf37acc2b2e1bc83eae341507cc3276b9da774d7e4f41facd): Bind for :::8080 failed: port is already allocated

Run 'docker run --help' for more information

```



Step8: Clean up experimental containers

Execute cleanup commands to recycle runtime resources:

```

bash
docker stop my-web-container second-container
docker rm my-web-container second-container
docker ps

```

`docker ps` outputs blank table, meaning all containers are completely stopped and deleted.

```

● @yi-xian0706 → /workspaces/Cloud-Computing (main) $ docker stop my-web-container second-container
my-web-container
second-container
● @yi-xian0706 → /workspaces/Cloud-Computing (main) $ docker rm my-web-container second-container
my-web-container
second-container
● @yi-xian0706 → /workspaces/Cloud-Computing (main) $ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
○ @yi-xian0706 → /workspaces/Cloud-Computing (main) $

```

Step9: Commit source files to GitHub repository via Git

```
bash
git add .
git commit -m "Finished Lab7 Docker Containerization Experiment"
git push
```

All modified files (Dockerfile, index.html) are synchronized to remote GitHub repository successfully.

```
● @yi-xian0706 → /workspaces/Cloud-Computing (main) $ git add .
● @yi-xian0706 → /workspaces/Cloud-Computing (main) $ git commit -m "Finished
  Lab7 Docker Containerization Experiment"
[main ae225d1] Finished Lab7 Docker Containerization Experiment
  2 files changed, 4 insertions(+), 1 deletion(-)
  create mode 100644 Dockerfile
● @yi-xian0706 → /workspaces/Cloud-Computing (main) $ git push
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 455 bytes | 227.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/yi-xian0706/Cloud-Computing
  00dc679..ae225d1  main -> main
○ @yi-xian0706 → /workspaces/Cloud-Computing (main) $ █
```

4. Core Experiment Conclusion

1. **Immutable Image Mechanism:** Built Docker image is read-only, local source file modification cannot affect running containers. New changes need rebuilding image and restarting container to take effect.
2. **Lightweight Feature:** Docker containers share host kernel, leading to ultra-low memory consumption (only several MB for Nginx container), supporting massive elastic deployment on single host.
3. **Network Port Rule:** Host port is exclusive resource; repeated binding will trigger port allocation failure, different ports are required for multi-instance expansion of same application image.
4. **File System Isolation:** Each container has independent isolated file system, inner files are separated from host files to avoid unexpected environment pollution.

5. Postscript

Convert this report document into PDF file named Lab7.pdf, then upload the PDF to learning platform and corresponding GitHub repository as required by the teacher.